

```

import numpy as np
import matplotlib.pyplot as plt

def resolver_gauss(A, b):
    """Implementação manual da Eliminação de Gauss com substituição retroativa[cite: 19, 20]."""
    n = len(b)
    # Matriz aumentada
    Ab = np.concatenate((A, b.reshape(n, 1)), axis=1)

    # Eliminação progressiva
    for i in range(n):
        # Pivoteamento parcial simples
        max_row = np.argmax(np.abs(Ab[i:n, i])) + i
        Ab[i], Ab[max_row] = Ab[max_row].copy(), Ab[i].copy()

        for j in range(i + 1, n):
            ratio = Ab[j, i] / Ab[i, i]
            Ab[j, i:] -= ratio * Ab[i, i:]

    # Substituição retroativa
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])) / Ab[i, i]
    return x

def analisar_trelica():
    print("--- Sistema de Análise de Treliças Planas ---")

    # 1. Entradas [cite: 9-16]
    # Usando a Treliça Warren de teste padronizada
    n_nos = 4
    n_barras = 5

    # Coordenadas (x, y) [cite: 11]
    # Exemplo: Triângulo (Nó 0: 0,0; Nó 1: 2,0; Nó 2: 1,1; Nó 3: 3,1)
    x = np.array([0.0, 2.0, 4.0, 2.0])
    y = np.array([0.0, 0.0, 0.0, 2.0])

    # Conectividade (de_no, para_no) [cite: 13, 14]
    barras = np.array([[0, 1], [1, 2], [0, 3], [1, 3], [2, 3]])

    # Cargas (Fx, Fy) em cada nó [cite: 15]

```

```

Fx = np.zeros(n_nos)
Fy = np.zeros(n_nos)
Fy[3] = -1000 # Carga vertical de 1000N no nó central superior [cite: 45]

# Apoios: 0=livre, 1=móvel(Y restrito), 2=fixo(X,Y restritos) [cite: 16]
apoios = [2, 1, 0, 0] # Fixo no esquerdo (0), Móvel no direito (1) [cite: 45]

# 2. Processamento: Matriz de Equilíbrio
# O sistema possui 2N equações (Soma Fx e Soma Fy para cada nó)
num_eq = 2 * n_nos
num_vars = n_barras + 3 # Barras + 3 reações de apoio (2 no fixo, 1 no móvel)

A = np.zeros((num_eq, num_vars))
b_vec = np.zeros(num_eq)

for i in range(n_nos):
    b_vec[2*i] = -Fx[i]
    b_vec[2*i + 1] = -Fy[i]

# Preenchimento da matriz com componentes das barras
for i in range(n_barras):
    n1, n2 = barras[i]
    dx = x[n2] - x[n1]
    dy = y[n2] - y[n1]
    L = np.sqrt(dx**2 + dy**2)
    cos, sin = dx/L, dy/L

    # Nó 1
    A[2*n1, i] = cos
    A[2*n1+1, i] = sin
    # Nó 2 (sentido oposto)
    A[2*n2, i] = -cos
    A[2*n2+1, i] = -sin

# Adicionando reações de apoio (simplificado para o teste padronizado)
A[0, n_barras] = 1 # Rx no Nó 0 (Fixo)
A[1, n_barras + 1] = 1 # Ry no Nó 0 (Fixo)
A[3, n_barras + 2] = 1 # Ry no Nó 1 (Móvel)

# 3. Resolver Sistema [cite: 19]
resultados = resolver_gauss(A, b_vec)
forcas = resultados[:n_barras]

```

```

# Classificação [cite: 21]
status = ["Tração (+)" if f > 0.1 else "Compressão (-)" if f < -0.1 else "Nula" for f in forcas]

# 4. Saídas Gráficas [cite: 23-28]
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# G1: Treliza Original [cite: 24]
for i, (n1, n2) in enumerate(barras):
    axs[0, 0].plot([x[n1], x[n2]], [y[n1], y[n2]], 'k-o')
for i in range(n_nos):
    axs[0, 0].text(x[i], y[i], f' {i}', color='red', fontweight='bold')
axs[0, 0].set_title("Treliza Original (Nós Numerados)")

# G2: Mapa de Calor (Tração/Compressão) [cite: 25, 26]
for i, (n1, n2) in enumerate(barras):
    cor = 'red' if forcas[i] < -0.1 else 'blue' if forcas[i] > 0.1 else 'gray'
    axs[0, 1].plot([x[n1], x[n2]], [y[n1], y[n2]], color=cor, lw=3)
axs[0, 1].set_title("Intensidade: Vermelho (-) / Azul (+)")

# G3: Magnitude das Forças [cite: 27]
axs[1, 0].bar(range(n_barras), np.abs(forcas), color='skyblue')
axs[1, 0].set_title("Magnitude da Força por Barra")
axs[1, 0].set_xlabel("Índice da Barra")

# G4: Histograma [cite: 28]
axs[1, 1].hist(forcas, bins=5, color='green', alpha=0.7)
axs[1, 1].set_title("Distribuição das Forças Internas")

plt.tight_layout()
plt.show()

# Estatísticas [cite: 22]
print(f"Barra mais solicitada: Barra {np.argmax(np.abs(forcas))} ({np.max(np.abs(forcas)):.2f} N)")
print(f"Força média: {np.mean(np.abs(forcas)):.2f} N")

analisar_treliza()

```